Desenvolvimento de um agente autónomo para o jogo
Arkanoid

Trabalho de Grupo

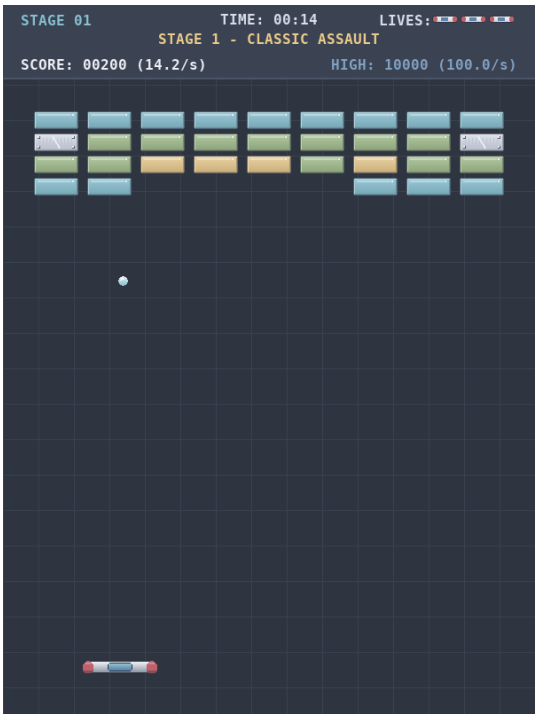
Inteligência Artificial

Ano Letivo de 2026/2027

Mário Antunes
mario.antunes@ua.pt

Luís Seabra Lopes
lsl@ua.pt

Outubro de 2026



I Notas importantes

1. Este trabalho deve ser realizado em grupos de três alunos. Os membros do grupo devem estar identificados pelo nome e número mecanográfico no `README.md` do respectivo repositório.
2. Uma primeira versão do programa deve ser submetida até 18 de novembro de 2026. A versão final deve ser submetida até 16 de dezembro de 2026. Em ambas as submissões, o trabalho pode ser submetido fora de prazo, mas será penalizado em 5% por cada dia adicional.
3. Cada grupo deve submeter o código no *GitHub* (repositório do grupo). Na submissão final, inclua uma apresentação (tipo Powerpoint) em formato `.pdf` com o nome `presentation.pdf`, com um máximo de cinco páginas, onde deve resumir a arquitetura do agente desenvolvido.
4. O código deve ser desenvolvido em Python 3.12, a versão por omissão usada para executar e avaliar os agentes (3.10 é o mínimo suportado). O módulo principal deve chamar-se `student.py`. O agente é avaliado em *modo de inferência* e só pode depender das bibliotecas declaradas em `pyproject.toml`: qualquer outra biblioteca (incluindo as usadas para treinar um modelo) não está instalada, pelo que o modelo treinado deve ser guardado em ficheiros do repositório e executado com essas bibliotecas.
5. Se discutir este trabalho com colegas de outros grupos, inclua um comentário com o nome e o número mecanográfico desses colegas. Se utilizar outras fontes, cite-as também.
6. Todo o código submetido deve ser original; embora se confie em que a maioria dos grupos cumprirá este requisito, serão utilizadas ferramentas de deteção de plágio. Os alunos envolvidos em casos de plágio verão o trabalho anulado.
7. O projeto será avaliado tendo em conta: desempenho; qualidade da arquitetura e da implementação; e originalidade.

II Introdução

Este trabalho envolve a aplicação de conceitos e técnicas dos principais capítulos da disciplina de IA, nomeadamente: programação em Python; arquiteturas de agentes; e técnicas de tomada de decisão e de resolução automática de problemas.

No âmbito deste trabalho, deve desenvolver um agente capaz de jogar de forma inteligente uma variante do jogo Arkanoid, um jogo de arcada desenvolvido pela Taito, no Japão, em 1986.

No *Arkanoid*, o jogador controla uma raquete (a *Vaus*) que desliza na horizontal ao longo da parte inferior do ecrã. Uma bola ressalta pelo campo de jogo e o jogador deve mantê-la em jogo, batendo-lhe com a raquete. De cada vez que a bola toca num tijolo, o tijolo é danificado e, quando a sua resistência chega a zero, parte-se e atribui pontos. Um nível está concluído quando todos os tijolos destrutíveis foram partidos.

O objetivo principal é concluir todos os níveis, acumulando pontos e sem perder a bola. Os tijolos têm resistências diferentes: quantos mais impactos um tijolo exige, mais pontos vale. Alguns tijolos são de metal, não podem ser destruídos e apenas desviam a bola. Falhar a bola custa uma vida; o jogo termina quando todas as vidas se perdem ou quando todos os níveis são concluídos.

Esta variante é deliberadamente simples de observar e difícil de dominar: o agente controla apenas a posição horizontal da raquete, mas o sítio onde a bola bate na raquete (terço esquerdo, central ou direito) decide para onde a bola segue. Um bom agente não se limita, portanto, a *seguir* a bola, também a *direciona*.

III Regras do jogo

- O campo de jogo tem 600×800 píxeis, com um teto em $y = 82$ (a zona acima é o HUD). Os tijolos têm 50×20 píxeis. A raquete tem 84×14 píxeis e a bola tem um raio de 6 píxeis.
- O jogador do *Arkanoid* (agente do aluno) tem acesso em todos os momentos a todo o estado: raquete, posição e velocidade da bola, todos os tijolos ativos, vidas, pontuação e tempo. A bola move-se a uma velocidade constante de 400 píxeis por segundo, com deteção contínua de colisões (não atravessa tijolos).
- O jogo corre a 30 ciclos por segundo de tempo real. Em cada ciclo o servidor envia o estado e o agente responde com uma ação: `{"action": "move", "direction": "WEST"}` ou `{"action": "move", "direction": "EAST"}`. Cada ação desloca a raquete 28 píxeis assim que é recebida; sem resposta, a raquete fica parada. O servidor não espera pelo agente, pelo que um agente lento reage apenas a menos estados.
- A cor dos tijolos indica quantos impactos exigem: vermelho (1 impacto, 50 pontos), azul (2 impactos, 100 pontos), verde (3 impactos, 150 pontos) e amarelo (4 impactos, 200 pontos). Os pontos são atribuídos *apenas quando um tijolo se parte*; danificar um tijolo não pontua. Os tijolos de metal são indestrutíveis, não pontuam e limitam-se a desviar a bola.
- A raquete está dividida em três zonas iguais que definem o ângulo de ressalto: o terço **central** envia a bola quase na vertical (um pequeno cone aleatório em torno de 90°); o terço **esquerdo** envia-a para a esquerda a cerca de 45° e o terço **direito** para a direita a cerca de 45°, cada um com uma pequena dispersão aleatória. A velocidade da bola nunca muda, apenas a sua direção.
- O jogo é composto por 3 níveis de dificuldade crescente, jogados em sequência (o primeiro tem uma parede clássica, o último tem colunas de metal e tijolos amarelos de 4 impactos). Um nível termina assim que não resta nenhum tijolo destrutível, mesmo que restem tijolos de metal. A bola é relançada a partir da raquete, com uma ligeira inclinação aleatória, no início de cada nível e após cada vida perdida.
- O agente tem 3 vidas por jogo. Um jogo termina (*game over*) quando a terceira vida se perde, e é ganho quando todos os níveis jogados são concluídos. Cada nível tem uma duração máxima de 400 segundos de tempo de jogo (tempo simulado, que não muda com o `--acceleration` do servidor); o relógio recomeça em cada nível. O jogo só passa ao nível seguinte se o atual for *concluído*; se a duração máxima for atingida primeiro, o jogo termina (`timed_out`) com a pontuação e o tempo acumulados até esse momento. Os níveis a jogar são escolhidos ao iniciar o servidor (parâmetro `--levels`, por exemplo `--levels 2` joga apenas o nível 2), e a duração máxima pode ser alterada com `--level-time-limit`.
- A pontuação bruta (`score`) é a soma dos pontos de todos os níveis (a *high score*). Serve para desempatar agentes com o mesmo valor de desempenho, e é tudo o que tem um agente que não conclui nenhum nível.
- A pontuação normalizada (`normalized_score`) é uma métrica por nível: os pontos obtidos no nível a dividir pelos segundos de tempo de jogo desse nível (o relógio recomeça em cada nível). Só conta se o nível for *concluído*: um nível perdido ou que esgota o tempo não ganha pontuação normalizada (0).
- A medida de desempenho é a pontuação normalizada média (`avg_norm_score`): a média da `normalized_score` sobre todos os níveis da sequência jogada, contando 0 para os níveis

não concluídos ou não alcançados. É o valor enviado ao servidor e usado na avaliação, e é mostrado no HUD.

IV Código e apoio ao desenvolvimento

O repositório contém o servidor (`server/`), um visualizador em Pygame (`viewer/`), agentes de exemplo (`agents/`), os ficheiros dos níveis (`maps/`) e as constantes do jogo (`config.json`). O `README.md` explica como criar o ambiente virtual, como instalar os requisitos (estão declarados em `pyproject.toml`; não existe `requirements.txt`: execute `pip install .` dentro do ambiente virtual ativo) e como iniciar o servidor, o visualizador e um agente.

Cada grupo desenvolve um agente criando um cliente que segue o protocolo exemplificado em `agents/dummy_agent.py`: crie uma subclasse de `BaseAgent` e implemente `deliberate()`, que lê `self.current_state` e devolve uma ação (ou `None`). O `agents/manual_agent.py` permite jogar com o teclado para ganhar sensibilidade ao jogo. Não são necessárias modificações aos ficheiros do servidor (não pode alterar `server/logic.py`), mas pode criar novos ficheiros, pastas, etc.

O estado enviado ao agente é um objeto JSON, do qual os campos seguintes são os mais úteis:

`paddle_x`, `paddle_width`	posição (extremo esquerdo) e largura da raquete
`ball_x`, `ball_y`, `ball_vx`, `ball_vy`	centro e velocidade da bola (píxeis, píxeis/s)
`bricks`	tijolos ativos: `x`, `y`, `type`, `hits_remaining`, `points`, `indestructible`
`lives`, `score`, `normalized_score`, `avg_norm_score`, `time_elapsed`	progresso do jogo
`stage`, `game_over`, `game_won`, `timed_out`	nível e fim do jogo
`level_time_elapsed`, `level_time_limit`	tempo usado e permitido no nível atual
`valid_actions`	movimentos permitidos neste tick

Se implementar uma nova funcionalidade ou qualquer melhoria ao motor do jogo e/ou ao visualizador, pode criar um *Pull Request* (PR) na plataforma GitHub. Se a sua alteração for aceite, receberá uma bonificação na avaliação final até um máximo de 1 valor (em 20).

O agente desenvolvido deve ser entregue num módulo com o nome `student.py` e deve ligar-se ao servidor de jogo local, usando como *id* (o argumento `--id`, ou a variável `AGENT_ID` do módulo) o número mecanográfico de um dos elementos do grupo (qualquer um). O *id* é enviado ao servidor e é o que identifica e classifica o grupo no *dashboard*; o argumento `--name` é apenas um nome para apresentação. Se não for indicado nenhum *id*, é gerado um aleatório, pelo que uma submissão sem *id* não pode ser classificada. O argumento `--id` tem precedência sobre `AGENT_ID`.

Existe um canal de apoio em `https://detiuaveiro.slack.com/messages/ai/` onde os alunos podem colocar dúvidas e receber notificações de alterações.

Dada a novidade do motor do jogo, é natural que venham a ser detectados alguns erros e introduzidos ajustes durante a realização do trabalho. São por isso esperadas várias atualizações do repositório *template*. Um *dashboard* com a classificação dos agentes será também disponibilizado oportunamente. Esteja atento a estas modificações (configure o repositório *template* como repositório *upstream* e faça *fetch*/*merge* com frequência) e a notificações por *e-mail*, *Slack* e *e-learning*.

V Avaliação

Os agentes serão avaliados numa *sequência de mapas*: os três níveis fornecidos no repositório e alguns mapas adicionais, que não são publicados. Cada jogo percorre os mapas da sequência por ordem (com 3 vidas para todo o jogo) e só passa ao mapa seguinte se o atual for concluído na sua duração máxima de 400 segundos. O valor de um jogo é a sua `avg_norm_score`: a média, sobre todos os mapas da sequência, dos pontos por segundo obtidos em cada mapa, contando 0 para os mapas não concluídos e para os mapas não alcançados. O jogo termina quando o agente perde a última vida, quando o mapa atual não é concluído na sua duração máxima, ou quando todos os mapas são concluídos. Um agente que não conclui nenhum mapa obtém 0, por muitos pontos que recolha; a pontuação bruta serve apenas para desempatar. Os valores de todos os jogos de um agente são combinados numa média única, pelo que um mapa fraco baixa o resultado final.

A `avg_norm_score` média de cada agente submetido é mapeada em notas tendo em conta a distribuição dessas médias. Um valor de 10 pontos por segundo é mapeado para 10/20, a mediana para 16/20 e o máximo para 20/20; os restantes valores são mapeados linearmente (0 é mapeado para 0/20). Não há outro mínimo: os pesos abaixo aplicam-se à nota de desempenho.

Avaliação da 1.ª entrega

- Nesta entrega, apenas o código do agente deve ser submetido.
- Cada agente jogará 10 jogos da sequência de mapas e será obtida a média desses jogos, a partir da qual se calcula a nota.

Avaliação da 2.ª entrega

- Cada agente jogará 10 jogos da sequência de mapas.
- Nesta entrega é necessário submeter uma apresentação (ver ponto I.3 acima).
- A avaliação da segunda entrega reflete o desempenho do agente (90%), a qualidade do desenho (de acordo com a apresentação entregue, 7,5%) e a qualidade da implementação (2,5%).

VI Recomendações

1. Comece por ler o `README.md` e seguir o seu arranque rápido *por ordem*: criar o ambiente virtual, ativá-lo e *depois* instalar os requisitos com `pip install .`. Ativar um ambiente virtual não instala nada, e a framework (`ai-game-framework`) não se instala à mão.
2. Estude depois `agents/dummy_agent.py` e `agents/base_agent.py`. O código é muito básico e simples, pelo que deve começar por fazer *refactoring* do cliente para algo mais orientado a um agente autónomo de IA.
3. Comece com um agente que se limita a seguir a bola e meça-o. Depois melhore-o: use a velocidade para prever onde a bola vai cair (incluindo ressaltos nas paredes) e escolha que terço da raquete usar para enviar a bola para onde parte os tijolos mais valiosos.
4. Use `--acceleration` e `--levels` para correr o servidor mais depressa, ou apenas num nível, enquanto desenvolve e compara versões (ver o README). Compare versões com vários jogos: o ângulo de lançamento é aleatório.
5. Faça periodicamente `git fetch` do repositório *template* para atualizar o seu código.

6. Execute `git log` para acompanhar as pequenas alterações que foram feitas.

7. Acompanhe o canal #ai no Slack.

VII Esclarecimento de dúvidas

Aqui serão colocados esclarecimentos sobre as principais dúvidas que possam surgir durante a realização do trabalho.

1. **Pergunta**: O `pip install` da framework falha com "Could not find a version that satisfies the requirement". O que está errado?

   Resposta: A framework é o pacote PyPI `ai-game-framework` (importado como `aigf`); não existe nenhum pacote chamado `aigf`. Não instale pacotes um a um: ative o ambiente virtual e execute `pip install .` na raiz do repositório, o que instala tudo o que está declarado em `pyproject.toml`.